MATH 415-01 Advanced Linear Algebra
MATLAB Primer

MATLAB is a useful program for performing matrix and linear algebra calculations. This document will outline some of the basic MATLAB operations we will use. Open MATLAB and try these commands yourself as you read through the document.

1. Enter a matrix
$$A = \begin{bmatrix} -1 & 2 & 4 \\ 0 & 7 & 11 \end{bmatrix}.$$
MATLAB lists out rows, splitting up rows with a semicolon. On the command line type

```
A = [[-1 2 4];[0 7 11]]
```

The command line is in the Command Window and indicated by `>>`. Note that in the Workspace window the variable `A` has been created.

Note that it does not use commas between entries. Compare the above with the same command that ends with a semicolon, which supresses the output. Try both.

```
A = [[-1 2 4];[0 7 11]];
```

Also create the matrix
$$B = \begin{bmatrix} 0 & -6 & -1 \\ 2 & 3 & 1 \\ 4 & 1 & 2 \end{bmatrix}$$
and the vector
$$\mathbf{x} = \begin{bmatrix} 3 \\ 2 \\ -1 \end{bmatrix}.$$
Vectors can be entered simply by

```
x = [3;2;-1]
```

2. Making a mistake.

   MATLAB does not allow you to modify a line that has been entered on the command line. If you make a mistake, use the up arrow on the keyboard to show a list of previous commands. Then select the line you want to bring back and fix your mistake. This is also a handy feature for bringing back previous commands without retyping them.

3. Matrix multiplication.

   Compute AB and BA by `A*B` and `B*A`. Try both on the command line. Note that `A*B` creates a variable `ans` in the Workspace window and the other entry generates an error message. We can always save the result of an action in a new variable. Compute $\mathbf{v} = B\mathbf{x}$ by `v = B*x`. Note that the variable `v` is created in the Workspace window.

4. Matrix transpose.

   The transpose is computed by `A'`. Try it. Also try BA^T by entering `B*A'`. This produces a matrix.

5. Solving a system.

   We can solve a system multiple ways.

   (a) Matrix inverse.
   We can solve $B\mathbf{x} = \mathbf{v}$ by $\mathbf{x} = B^{-1}\mathbf{v}$, if B has an inverse. Try `B^(-1)*v`.

(b) RREF

We can row reduce the system by `rref([B v])`. Note that we defined a new matrix using `[B v]` inside the `rref` command. This is a handy way of building matrices out of other matrices and vectors.

(c) `mldivide` command

MATLAB has a special command for solving systems. We can either type `mldivide(B,v)` or use the shortcut command `B\v`. Try both. This is a handy command because it checks the system for various properties and then chooses the optimal method for solving the system. If I need a RREF matrix, then I will use `rref`, but to solve a system I use the backslash shortcut.

6. Determinant.

   The determinant of B is simply `det(B)`. Try `det(A)`, which produces an error.

7. Eigenvalues and eigenvectors.

   `e_vals = eig(B)` returns a column vector of the eigenvalues of B.

   We can get two outputs from the `eig` command that gives us both the eigenvalues and the eigenvectors of B. To get both outputs we have to remember that this is a matrix laboratory, and MATLAB loves matrices, so we need to specify an output that is a matrix. We do this like we did for solving a system of equations by typing `[e_vecs, e_vals] = eig(B)`. Now `e_vals` is a diagonal matrix with the eigenvalues on the diagonal. This means that the `eig` command is useful for diagonalizing a matrix, because it provides us both the diagonal matrix D with the eigenvalues, as well as the invertible matrix P of eigenvectors, so that $B = PDP^{-1}$.

   Type `[P,D] = eig(B)`. Then type `P*D*P^(-1)`, which should return B. You'll have to read the result carefully, because MATLAB is going to return complex answers because the inputs had complex entries, but note that the imaginary part of each entry is 0.

8. Dot product.

   We don't need a special command for the dot product, since $\mathbf{u} \cdot \mathbf{v} = \mathbf{u}^T\mathbf{v}$. If we want $\mathbf{x} \cdot \mathbf{v}$, we can type `x'*v`. That said, we can also use `dot(x,v)`.